Naive String Matching

What is String matching??
Ans: Finding all occurrences of a pattern in a given text(or body of text).

- Naive pattern searching is the simplest method among other pattern searching algorithms. It checks for all characters of the main string to the pattern.
- Naive algorithm is exact string matching(means finding one or all exact occurrences of a pattern in a text) algorithm.
- This algorithm is helpful for smaller texts. It does not need any pre-processing phases. We can find substring by checking once for the string. It also does not occupy extra space to perform the operation.
- The naive approach tests all the possible placement of Pattern P [1…….m] relative to text T [1……n]. We try shift s = 0, 1…….n-m, successively and for each shift s. Compare T [s+1…….s+m] to P [1……m].It returns all the valid shifts found.

NAIVE-STRING-MATCHER (T, P)
1. n ← length [T]
2. m ← length [P]
3. for s ← 0 to n -m
4. do if P [1.....m] = T [s + 1....s + m]
5. then print "Pattern occurs with shift"

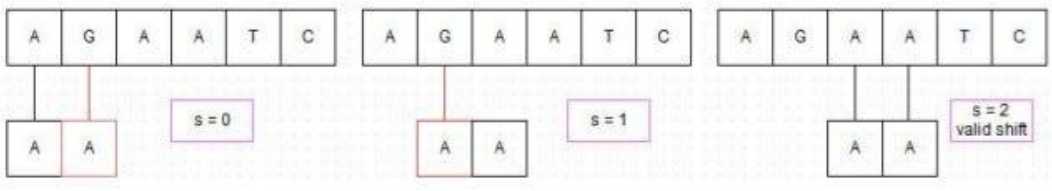
Analysis: This for loop from 3 to 5 executes for n-m + 1(we need at least m characters at the end) times and in iteration we are doing m comparisons. So the total complexity is O (n-m+1).

- The test on line 4 determines whether the current shift is valid or not;this test involves an implicit loop to check corresponding character positions until all positions.
- Line 5 prints out each valid shift s.

Working of Naive String Matching

The naive-string-matching procedure can be interpreted graphically as sliding a “template” containing the pattern over the text, noting for which shifts all of the characters on the template equal the corresponding characters in the text.

Example 1:



Example 2:

Input: txt[] = "THIS IS STRING MATCHING ALGORITHM"

pat[] = "STRING"

Output: Pattern found at position 10

Example 3:

Input: Main String: “ABAAABCDBBABCDDEBCABC”
pattern: “ABC”

Output:
Pattern found at position: 4
Pattern found at position: 10
Pattern found at position: 18

What is the best case?
→The best case occurs when the first character of the pattern is not present in text at all.
txt[] = "BBACCAADDEE";
pat[] = "HBB";
The number of comparisons in best case is O(n).

What is the worst case ?
→The worst case of Naive Pattern Searching occurs in following scenarios.

1) When all characters of the text and pattern are same.
txt[] = "DDDDDDDDDDDD";
pat[] = "DDDDD";

2) Worst case also occurs when only the last character is different.
txt[] = "VVVVVVVVVVVVK";
pat[] = "VVVK";

The number of comparisons in the worst case is O(m*(n-m+1)).

Advantages:

- No Pre-processing phase required because the running time of Naive-String-Matcher is equal to its matching time.
- No extra space are needed.
- Also,the comparisons can be done in any order.

Disadvantage:

- Naive method is inefficient because information from a shift is not used again.
- The naive algorithm finds all valid shifts using a loop that checks the condition P [1 . . m] = T[s + 1 . . s + m] for each of the n - m + 1 possible values of s.

Rabin Karp Algorithm

Rabin-Karp algorithm is an algorithm used for searching/matching patterns in the text using a hash function. Unlike Naive string matching algorithm, it does not travel through every character in the initial phase, rather it filters the characters that do not match and then performs the comparison.
A hash function is a tool to map a larger input value to a smaller output value. This output value is called the hash value.

How Rabin-Karp Algorithm Work?

A sequence of characters is taken and checked for the possibility of the presence of the required string. If the possibility is found then, character matching is performed.

Let us understand the algorithm with the following steps:

Let the text be:

And the string to be searched in the above text be:

Let us assign a numerical value(v)/weight for the characters we will be using in the problem. Here, we have taken first ten alphabets only (i.e. A to J).

n be the length of the pattern and m be the length of the text. Here, m = 10 and n = 3.

Let d be the number of characters in the input set. Here, we have taken the input set {A, B, C, ..., J}. So, d = 10. You can assume any suitable value for d.

Let us calculate the hash value of the pattern.

hash value for pattern(p) = Σ(v * dm-1) mod 13

= ((3 * 102) + (4 * 101) + (4 * 100)) mod 13

= 344 mod 13

= 6

In the calculation above, choose a prime number (here, 13) in such a way that we can perform all the calculations with single-precision arithmetic.

The reason for calculating the modulus is given below.

Calculate the hash value for the text-window of size m.

For the first window ABC,

hash value for text(t) = Σ(v * dn-1) mod 13

= ((1 * 102) + (2 * 101) + (3 * 100)) mod 13

= 123 mod 13

= 6

Compare the hash value of the pattern with the hash value of the text. If they match then, character-matching is performed.

In the above examples, the hash value of the first window (i.e. t) matches with p so, go for character matching between ABC and CDD. Since they do not match so, go for the next window.

We calculate the hash value of the next window by subtracting the first term and adding the next term as shown below.

t = ((1 * 102) + ((2 * 101) + (3 * 100)) * 10 + (3 * 100)) mod 13

= 233 mod 13

= 12

In order to optimize this process, we make use of the previous hash value in the following way.

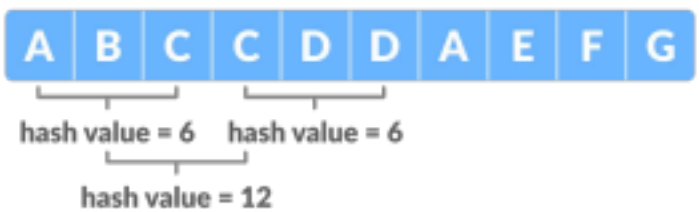
t = ((d * (t - v[character to be removed] * h) + v[character to be added]) mod 13 = ((10 * (6 - 1

* 9) + 3)mod 13

= 12

Where, h = dm-1 = 103-1 = 100.
For BCC, t = 12 (≠6). Therefore, go for the next window.

After a few searches, we will get the match for the window CDA in the text.



Algorithm:

```
RABIN-KARP-MATCHER(T, P, d, q)
n = T.length
m = P.length
h = d^(m-1) mod q
p = 0
t_0 = 0
for i = 1 to m                    // preprocessing
    p = (dp + P[i]) mod q
    t_0 = (dt_0 + T[i]) mod q
for s = 0 to n - m                // matching
    if p == t_s
        if P[1..m] == T[s + 1..s + m]
            print "Pattern occurs with shift" s
    if s < n - m
        t_{s+1} = (d(t_s - T[s + 1]h) + T[s + m + 1]) mod q
```

Limitations of Rabin-Karp Algorithm

1. Spurious Hit

When the hash value of the pattern matches with the hash value of a window of the text but the window is not the actual pattern then it is called a spurious hit.

Spurious hit increases the time complexity of the algorithm. In order to minimize spurious hit, we use modulus. It greatly reduces the spurious hit.

Rabin-Karp Algorithm Complexity

The average case and best case complexity of Rabin-Karp algorithm is O(m + n) and the worst case complexity is O(mn).

The worst-case complexity occurs when spurious hits occur a number for all the windows.

Rabin-Karp Algorithm Applications

- For pattern matching
- For searching string in a bigger text

Knuth-Morris-Pratt Algorithm

KMP Algorithm is one of the most popular patterns matching algorithms. KMP stands for Knuth Morris Pratt. KMP algorithm was invented by **Donald Knuth** and **Vaughan Pratt** together and independently by **James H Morris** in the year 1970. In the year 1977, all the three jointly published the KMP Algorithm.

KMP algorithm was the first **linear time complexity** algorithm for string matching. KMP

algorithm is one of the string matching algorithms used to find a Pattern in a Text.

KMP algorithm is used to find a **"Pattern"** in a **"Text"**. This algorithm compares character by character from left to right. But whenever a mismatch occurs, it uses a preprocessed table called **"Prefix Table"** to skip characters comparison while matching. Sometimes the prefix table is also known as **LPS Table**. Here LPS stands for **"Longest proper Prefix which is also Suffix"**.

Algorithm:

```
KMP-MATCHER(T, P)
n = T.length
m = P.length
π = COMPUTE-PREFIX-FUNCTION(P)
q = 0                                   // number of characters matched
for i = 1 to n                          // scan the text from left to right
    while q > 0 and P[q + 1] ≠ T[i]
        q = π[q]                        // next character does not match
    if P[q + 1] == T[i]
        q = q + 1                       // next character matches
    if q == m                           // is all of P matched?
        print "Pattern occurs with shift" i − m
        q = π[q]                        // look for the next match
```

```
COMPUTE-PREFIX-FUNCTION(P)
m = P.length
let π[1..m] be a new array
π[1] = 0
k = 0
for q = 2 to m
    while k > 0 and P[k + 1] ≠ P[q]
        k = π[k]
    if P[k + 1] == P[q]
        k = k + 1
    π[q] = k
return π
```

Steps for Creating LPS Table (Prefix Table)

Step 1 - Define a one dimensional array with the size equal to the length of the Pattern. (LPS[size])

Step 2 - Define variables **i & j**. Set i = 0, j = 1 and LPS[0] = 0.

Step 3 - Compare the characters at **Pattern[i]** and **Pattern[j].**

Step 4 - If both are matched then set **LPS[j] = i+1** and increment both i & j values by one. Goto to Step 3.

Step 5 - If both are not matched then check the value of variable 'i'. If it is '0' then set **LPS[j] = 0** and increment 'j' value by one, if it is not '0' then set **i = LPS[i-1]**. Goto Step 3.

Step 6- Repeat above steps until all the values of LPS[] are filled.

Let us use the above steps to create a prefix table for a pattern...

Example for creating KMP Algorithm's LPS Table (Prefix Table)

Consider the following Pattern

	0	1	2	3	4	5	6
Pattern :	A	B	C	D	A	B	D

Let us define LPS[] table with size 7 which is equal to length of the Pattern

	0	1	2	3	4	5	6
LPS							

Step 1 - Define variables i & j. Set i = 0, j = 1 and LPS[0] = 0.

	0	1	2	3	4	5	6
LPS	0						

i = 0 and j = 1

Step 2 - Campare Pattern[i] with Pattern[j] ===> A with B.
Since both are not matching and also "i = 0", we need to set LPS[j] = 0 and increment 'j' value by one.

	0	1	2	3	4	5	6
LPS	0	0					

i = 0 and j = 2

Step 3 - Campare Pattern[i] with Pattern[j] ===> A with C.
Since both are not matching and also "i = 0", we need to set LPS[j] = 0 and increment 'j' value by one.

	0	1	2	3	4	5	6
LPS	0	0	0				

i = 0 and j = 3

Step 4 - Campare Pattern[i] with Pattern[j] ===> A with D.
Since both are not matching and also "i = 0", we need to set LPS[j] = 0 and increment 'j' value by one.

	0	1	2	3	4	5	6
LPS	0	0	0	0			

i = 0 and j = 4

Step 5 - Campare Pattern[i] with Pattern[j] ===> A with A.
Since both are matching set LPS[j] = i+1 and increment both i & j value by one.

	0	1	2	3	4	5	6
LPS	0	0	0	0	1		

i = 1 and j = 5

Step 6 - Campare Pattern[i] with Pattern[j] ===> B with B.
Since both are matching set LPS[j] = i+1 and increment both i & j value by one.

	0	1	2	3	4	5	6
LPS	0	0	0	0	1	2	

i = 2 and j = 6

Step 7 - Campare Pattern[i] with Pattern[j] ===> C with D.
Since both are not matching and i !=0, we need to set i = LPS[i-1]
===> i = LPS[2-1] = LPS[1] = 0.

	0	1	2	3	4	5	6
LPS	0	0	0	0	1	2	

i = 0 and j = 6

Step 8 - Campare Pattern[i] with Pattern[j] ===> A with D.
Since both are not matching and also "i = 0", we need to set LPS[j] = 0 and increment 'j' value by one.

	0	1	2	3	4	5	6
LPS	0	0	0	0	1	2	0

Here LPS[] is filled with all values so we stop the process. The final LPS[] table is as follows...

	0	1	2	3	4	5	6
LPS	**0**	**0**	**0**	**0**	**1**	**2**	**0**

How to use LPS Table

We use the LPS table to decide how many characters are to be skipped for comparison when a mismatch has occurred.
When a mismatch occurs, check the LPS value of the previous character of the mismatched character in the pattern. If it is '0' then start comparing the first character of the pattern with the next character to the mismatched character in the text. If it is not '0' then start comparing the character which is at an index value equal to the LPS value of the previous character to the mismatched character in pattern with the mismatched character in the Text.

How the KMP Algorithm Works

Let us see a working example of KMP Algorithm to find a Pattern in a Text...

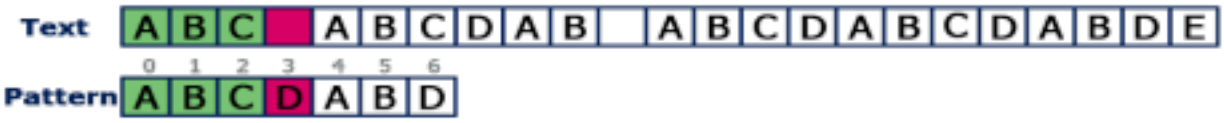
Consider the following Text and Pattern

Text : ABC ABCDAB ABCDABCDABDE
Pattern : ABCDABD

LPS[] table for the above pattern is as follows...

	0	1	2	3	4	5	6
LPS	0	0	0	0	1	2	0

Step 1 - Start comparing first character of Pattern with first character of Text from left to right



Here mismatch occured at Pattern[3], so we need to consider LPS[2] value. Since LPS[2] value is '0' we must compare first character in Pattern with next character in Text.

Step 2 - Start comparing first character of Pattern with next character of Text.

Here mismatch occured at Pattern[6], so we need to consider LPS[5] value. Since LPS[5] value is '2' we compare Pattern[2] character with mismatched character in Text.

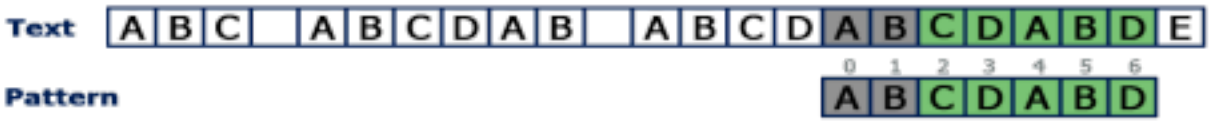
Step 3 - Since LPS value is '2' no need to compare Pattern[0] & Pattern[1] values

Here mismatch occured at Pattern[2], so we need to consider LPS[1] value. Since LPS[1] value is '0' we must compare first character in Pattern with next character in Text.

Step 4 - Compare Pattern[0] with next character in Text.

Here mismatch occured at Pattern[6], so we need to consider LPS[5] value. Since LPS[5] value is '2' we compare Pattern[2] character with mismatched character in Text.

Step 5 - Compare Pattern[2] with mismatched character in Text.



Here all the characters of Pattern matched with a substring in Text which is starting from index value 15. So we conclude that given Pattern found at index 15 in Text.

Complexity Analysis of KMP Algorithm

For analyzing the time complexity of the KMP string matching algorithm, let's define a variable k that will hold the value of i - j.

$k \leftarrow i - j$

So, for every iteration through the while loop, we can expect one of three things to occur:

- We find a match, and hence increment both i and j, hence k remains the same
- There is a mismatch but $j > 0$, so i does not change and hence we can say that k increases by at least 1 since k changes from i - j to i - LPS[j - 1]
- There is a mismatch but this time $j = 0$, hence i is incremented and k increases by 1.

So, each iteration of the loop, either i increases by 1, or k increases by 1. Hence the greatest possible number of loops would be 2n.

Now talking about the LPS compute function, our table is computed in linear time complexity so we can say that the time complexity is O(m).

Hence, the total time complexity of our KMP algorithm is O(n + m).